

Elasticsearch 搜索引擎高级面试题 完整答案

从倒排索引、查询相关性到集群治理与搜索架构

高级 Java 开发面试复习讲义

2026-06

阅读说明

本讲义面向具有 Java 后端、微服务或搜索平台经验的高级开发工程师。内容不是只罗列关键词的“八股答案”，而是按照真实面试中的表达顺序组织：**先给结论，再解释机制，然后给出配置或请求示例，最后讨论生产边界、排障方法和方案取舍。**

版本语境

- 截至 2026 年 6 月，Elastic 官方下载页列出的当前 Elasticsearch 版本为 9.4.2。本讲义以 **Elasticsearch 8.19/9.x 的通用语义**为主，同时兼顾大量生产环境仍在使用的 7.x/8.x。
- Elasticsearch 的底层搜索能力来自 Apache Lucene。涉及 segment、倒排索引、BM25、合并等内容时，应区分“Lucene 的底层机制”和“Elasticsearch 的分布式封装”。
- 文中的默认值以自建 Elastic Stack 的常见默认行为为主；Elastic Cloud Serverless 的部分默认值和可配置项不同，例如刷新间隔可能不同。
- 题目中的容量数字只是估算示例。真实选型必须使用生产数据、查询分布和目标 SLA 做基准测试。

建议的面试回答结构

面对原理题，可以按以下四步回答：

1. **一句话结论**：先表明判断，避免从细节开始绕圈。
2. **核心链路**：说明数据结构、请求流转或状态变化。
3. **边界与反例**：指出什么时候不成立、有什么副作用。
4. **生产实践**：给出监控指标、排障命令和选型依据。

面对系统设计题，建议按“目标与约束 → 数据模型 → 写入链路 → 查询链路 → 一致性 → 容量与容灾 → 可观测性 → 取舍”展开。

目录

1. Elasticsearch 与传统数据库的核心差异，倒排索引如何工作
2. Elasticsearch 为什么是近实时搜索，refresh、flush、translog、segment 有什么关系
3. Mapping 应如何设计，text、keyword、analyzer、multi-fields 如何选择
4. term、match、bool、filter 查询有什么区别
5. Elasticsearch 如何计算相关性，BM25 应如何调优
6. 分片、副本和路由是什么，写入与查询请求如何执行
7. 主分片数量应如何规划，如何估算分片与集群容量
8. from/size、search_after、PIT、scroll 应如何选择
9. 聚合为什么容易消耗内存，doc_values、fielddata 和全局序数是什么
10. object、nested、flattened 有什么区别，如何避免 Mapping Explosion
11. 如何提高批量写入吞吐量，同时避免把集群写垮
12. Elasticsearch 查询突然变慢，如何系统排查
13. 集群变成 yellow 或 red，未分配分片如何定位和恢复
14. JVM 堆、文件系统缓存、GC 和熔断器应如何理解
15. Mapping 不能直接修改时，如何使用 alias + reindex 实现无停机迁移
16. Elasticsearch 如何处理写入一致性、并发更新和版本冲突
17. 日志与时序数据为什么适合 Data Stream + ILM，如何设计生命周期
18. Snapshot 是否等于实时备份，如何设计备份与灾难恢复
19. 向量检索、BM25 和混合检索如何选型
20. 场景设计：构建一个高并发商品搜索系统
21. 附录：常用排障 API、容量估算、面试追问与参考资料

第一部分：基础原理

1. Elasticsearch 与传统数据库的核心差异，倒排索引如何工作？

1.1 结论先行

Elasticsearch 是面向搜索与分析的分布式数据引擎，核心优势是全文检索、相关性排序、聚合分析和横向扩展；关系型数据库更擅长事务、约束、精确更新和复杂关系计算。Elasticsearch 不应被简单理解为“更快的 MySQL”，两者解决的问题不同。

全文检索快的关键是**倒排索引**：它不是从文档逐条扫描关键词，而是提前建立“词项 → 包含该词项的文档集合”的映射。

1.2 正排索引与倒排索引

假设有三篇文档：

```
D1: Java 高并发开发
D2: Elasticsearch 搜索开发
D3: Java 搜索系统
```

正排视角是：

```
D1 -> [Java, 高并发, 开发]
D2 -> [Elasticsearch, 搜索, 开发]
D3 -> [Java, 搜索, 系统]
```

倒排视角是：

```
Java      -> [D1, D3]
搜索      -> [D2, D3]
开发      -> [D1, D2]
高并发    -> [D1]
Elasticsearch -> [D2]
系统      -> [D3]
```

实际 Lucene 倒排索引还会保存：

- 文档频率：一个词出现在多少文档中。
- 词频：一个词在某篇文档中出现几次。
- position：词在文档中的位置，用于短语查询。
- offset：字符偏移，用于高亮。
- skip information：帮助快速跳过不可能匹配的文档区间。

因此查询“Java 搜索”时，系统可以直接取两个倒排列表并求交集或并集，而不必扫描所有 `_source`。

1.3 分词链路

文本写入 `text` 字段时通常经过 analyzer：

```
character filter -> tokenizer -> token filter
```

例如：

```
"The QUICK Brown-Foxes"
```

可能得到：

```
[quick, brown, fox]
```

搜索时也会对查询文本进行分析。**索引时分析和查询时分析必须语义兼容**，否则文档明明包含用户输入，却无法匹配。

中文场景需要特别关注：

- 中文没有天然空格边界，标准分词器通常不能满足业务搜索需求。
- 业务中常使用 IK、SmartCN 或自定义词典，但插件版本必须与 Elasticsearch 版本匹配。
- 同义词、品牌名、型号和中英文混合词应通过离线词典和搜索分析器治理，而不是把所有规则塞进应用代码。

1.4 Elasticsearch 与数据库的典型差异

维度	Elasticsearch	关系型数据库
核心索引	倒排索引、BKD、doc values、向量索引	B+ 树、哈希索引等
查询目标	全文召回、相关性、过滤、聚合	精确查询、连接、事务计算
一致性	搜索近实时；单文档写有并发控制	强事务和约束能力更成熟
Schema	Mapping，可动态扩展但需治理	表结构与约束严格
更新成本	更新本质上生成新版本并标记旧文档删除	常见为页内或索引维护
横向扩展	原生分片与副本	依赖数据库分库分表或分布式实现
关系模型	倾向反范式、文档内聚	规范化、JOIN 能力强

1.5 为什么不能把 Elasticsearch 当唯一主库

不是绝对不能，而是多数交易系统不应这样做，原因包括：

- 不提供与传统数据库同等级的多行 ACID 事务。
- 唯一性、外键、复杂约束通常需要应用保证。
- 搜索可见性受 refresh 影响。
- Mapping 变更经常需要新建索引并 reindex。
- 错误删除或错误更新需要依赖快照、事件重放或上游主数据恢复。

常见架构是：

```
MySQL / PostgreSQL (事实主库)
    |
    CDC / BinLog / MQ
    |
Elasticsearch (查询与检索副本)
```

1.6 面试易错点

误区一：Elasticsearch 完全没有 Schema。

错误。它有 Mapping。动态映射只是帮助自动创建字段，并不等于没有类型约束。

误区二：倒排索引只能做字符串搜索。

不准确。Elasticsearch 对数值、日期、地理位置、向量等字段使用不同的数据结构；倒排索引只是全文与词项检索的核心之一。

追问：为什么 LIKE ‘%keyword%’ 通常不如倒排索引？

前导通配符难以利用传统 B+ 树的有序前缀，常常需要扫描大量值；倒排索引在写入时已经按词项组织文档集合。但 wildcard/regexp 查询本身也可能昂贵，不能因为用了 ES 就无条件高效。

2. Elasticsearch 为什么是近实时搜索？refresh、flush、translog、segment 有什么关系？

2.1 结论先行

文档写入成功并不一定立即能被 Search API 搜到。写入先进入内存索引缓冲区和 translog；refresh 会产生可搜索的新 segment，因此实现“近实时”；flush 则执行 Lucene commit 并切换 translog generation，主要服务于持久化边界和恢复，不等于让数据可搜索。

核心区别：

- **refresh**：让新数据可搜索。
- **translog fsync**：提升写入持久性。
- **flush**：Lucene commit + 新 translog generation。
- **segment merge**：合并不可变 segment，回收删除文档和减少查询开销。

2.2 写入后的状态变化

一次写入可以简化为：

```
请求到达主分片
-> 写入内存 indexing buffer
-> 追加 translog
-> 同步到副本分片
-> 根据 durability 策略 fsync
-> 返回响应
-> 后续 refresh 生成可搜索 segment
-> 后台 merge 合并 segment
-> 后续 flush 形成 Lucene commit
```

这里需要注意：真实实现还包含序列号、复制、检查点和恢复机制，面试不必把流程说成单机日志系统，但应明确“搜索可见性”和“写入持久性”是两个不同维度。

2.3 Segment 为什么不可变

Lucene segment 一旦生成就不可修改。更新文档时会：

1. 将旧文档标记为删除。
2. 写入一份新文档。
3. 后续 merge 时真正清理旧版本。

不可变带来的优点：

- 无需对 segment 内部结构做复杂并发更新。
- 文件可以被操作系统高效缓存。
- 搜索可以使用稳定的倒排结构。
- segment 可以复用和增量快照。

代价是：

- 高频更新会产生删除标记和新 segment。
- segment 过多会增加搜索和文件句柄开销。

- merge 会消耗 CPU、磁盘 IO 和临时磁盘空间。

2.4 refresh 与 refresh 参数

自建 Elastic Stack 常见默认 `index.refresh_interval` 为 1 秒。它不是严格定时承诺，而是近实时机制。

写入 API 的 refresh 典型取值：

- `false`：默认，不执行额外 refresh 动作。
- `wait_for`：等待自然 refresh 使本次变更可见，通常比强制 refresh 更友好。
- `true`：立即刷新相关分片，可能制造大量小 segment，降低持续写入吞吐量。

```
POST products/_doc/1001?refresh=wait_for
{
  "name": "Java 性能优化"
}
```

生产中不要对每次写入都设置 `refresh=true`。对于“写后立刻查询”的交互，可考虑：

- 直接使用实时 GET 按 `_id` 读取。
- 使用 `refresh=wait_for`。
- 业务侧返回刚写入的数据，不立即回查搜索索引。
- 对必须强可见的少量流程做单独索引或批次刷新。

2.5 translog 与 durability

Lucene commit 不会在每次写入后执行，因此 translog 用于恢复最近尚未 commit 的操作。节点重启时，可以从上一个安全提交点恢复 segment，再重放 translog。

常见 durability 语义是请求返回前进行 translog fsync，从而降低已确认写入在崩溃中丢失的风险。调低持久性可以提高吞吐，但会扩大故障时的数据丢失窗口，不能只看 benchmark。

2.6 flush 与 refresh 的区别

操作	主要目的	是否让新数据可搜索	典型成本
refresh	打开新的搜索视图	是	产生新 segment，频繁执行会降低吞吐
flush	Lucene commit、切换 translog	通常不是它的主要语义	磁盘提交与恢复边界
fsync translog	持久化最近写操作	否	磁盘同步写
merge	合并 segment、清理删除	不用于立即可见	CPU、IO、临时空间

2.7 merge 的生产风险

merge 通常自动调度和限速。常见问题：

- 大量更新或短 refresh 产生许多小 segment。
- 磁盘空间不足，merge 需要额外临时空间。
- 强制 force merge 活跃写索引，导致 IO 峰值和大 segment 后续继续被更新。
- 热点分片同时承担写入、搜索和 merge。

force merge 更适合不再写入的只读历史索引，并且应控制并发和磁盘余量。

2.8 面试追问

为什么刚写入 GET 能看到，Search 却看不到？

GET by ID 默认可以实时读取最近操作，Search 依赖 refresh 后的可搜索 segment，因此两者可见性不同。

refresh 越频繁，实时性越好，为什么不设为 100ms？

更频繁 refresh 会产生更多小 segment、增加打开搜索器和 merge 的成本，使写吞吐、搜索稳定性和磁盘 IO 恶化。实时性是有成本的 SLA，而不是免费参数。

3. Mapping 应如何设计？text、keyword、analyzer、multi-fields 如何选择？

3.1 结论先行

Mapping 设计决定字段如何被索引、查询、排序和聚合。text 用于分词全文检索，keyword 用于精确匹配、排序、聚合和去重。一个字段既要全文检索又要精确聚合时，使用 multi-fields，而不是在 text 上开启 fielddata。

3.2 text 与 keyword

需求	推荐字段类型
商品标题全文检索	text
商品 ID、订单号、状态码	keyword
用户标签精确过滤	keyword
描述正文	text 或 match_only_text（视版本与需求）
排序、terms 聚合	keyword、数值、日期等支持 doc values 的类型
前缀/输入联想	search_as_you_type、completion 或合理的 edge n-gram

示例：

```
PUT products-v1
{
  "mappings": {
    "dynamic": "strict",
    "properties": {
      "product_id": { "type": "keyword" },
      "title": {
        "type": "text",
        "analyzer": "standard",
        "fields": {
          "keyword": {
            "type": "keyword",
            "ignore_above": 256
          }
        }
      },
      "price": { "type": "scaled_float", "scaling_factor": 100 },
      "status": { "type": "keyword" },
      "created_at": { "type": "date" }
    }
  }
}
```

查询：

```
GET products-v1/_search
{
  "query": {
    "match": { "title": "Java 搜索" }
  },
  "sort": [
    { "title.keyword": "asc" }
  ]
}
```

3.3 index analyzer 与 search analyzer

- analyzer: 索引和默认搜索分析器。
- search_analyzer: 查询时使用, 可与索引时不同。
- search_quote_analyzer: 处理短语查询时可单独配置。

例如自动补全可能索引时使用 edge n-gram, 查询时使用普通分词器:

```
"name": {
  "type": "text",
  "analyzer": "autocomplete_index",
  "search_analyzer": "standard"
}
```

索引分析器负责产生足够的召回词项, 搜索分析器避免把用户输入切得过碎。两者不对称是允许的, 但必须通过 _analyze 验证实际 token。

```
POST products-v1/_analyze
{
  "field": "title",
  "text": "Elasticsearch 高级开发"
}
```

3.4 为什么不应滥用动态映射

默认动态映射可能把字符串同时映射为 text 和 keyword, 带来:

- 不需要的倒排索引和 doc values。
- 字段数量快速膨胀。
- 日期、数字被错误识别后难以修改。
- 不同写入批次产生类型冲突。
- 集群状态中的 Mapping 体积增大。

生产建议:

- 核心业务索引使用显式 Mapping 或 dynamic: strict。
- 半结构化扩展字段使用 dynamic template。
- 完全不需要搜索的对象设置 enabled: false。
- 大量任意键值优先评估 flattened, 不要每个 key 都创建字段。

3.5 不能直接修改字段类型怎么办

已有字段从 text 改成 keyword、从 long 改成 date, 通常不能原地修改, 因为底层索引结构已经生成。

常见流程:

1. 创建新索引和正确 Mapping。
2. 通过 _reindex 复制数据。

3. 校验数量、抽样、查询结果和业务增量。
4. 原子切换 alias。
5. 观察后删除旧索引。

这也是为什么 Mapping 设计是上线前的架构工作，而不是“先动态创建，错了再改”。

3.6 常见误区

在 text 字段上排序为什么报错或很慢？

text 的倒排结构面向词项检索，不适合按原值排序。应使用 keyword 子字段。开启 fielddata 会把词项结构加载进堆，内存代价很高。

数字 ID 应该使用 long 还是 keyword？

如果只做精确查询，不做范围和数学运算，keyword 往往更符合语义；需要范围查询时用数值类型。也可以 multi-field 双写，但会增加存储。

_source 是否等于索引？

不是。_source 是原始 JSON 的存储形式，搜索依赖倒排索引、doc values 等结构。关闭 _source 会影响更新、reindex、调试和部分功能，除非明确权衡，否则不要轻易关闭。

4. term、match、bool、filter 查询有什么区别？

4.1 结论先行

- term 是词项级查询，通常不分析输入，适合 keyword、数值、状态等精确值。
- match 是全文查询，会先使用字段的搜索分析器处理输入，再生成底层词项查询。
- query context 计算 _score，filter context 只判断是否匹配，通常更适合结构化条件，也更容易缓存和优化。
- bool 用于组合 must、filter、should、must_not。

4.2 term 与 match 示例

假设 title 是 text，写入：

```
{ "title": "Quick Brown Fox" }
```

标准分析后可能存储词项：

```
quick, brown, fox
```

下面的查询可能匹配不到：

```
GET docs/_search
{
  "query": {
    "term": { "title": "Quick Brown Fox" }
  }
}
```

因为 term 不会把输入分析为多个小写词项。应使用：

```
GET docs/_search
{
  "query": {
    "match": { "title": "Quick Brown Fox" }
  }
}
```

精确过滤应查询 keyword 字段：

```
"term": { "status": "PUBLISHED" }
```

4.3 bool 查询语义

```
GET products/_search
{
  "query": {
    "bool": {
      "must": [
        { "match": { "title": "Java 搜索" } }
      ],
      "filter": [
        { "term": { "status": "ONLINE" } },
        { "range": { "price": { "gte": 50, "lte": 300 } } }
      ],
      "must_not": [
```

```
{ "term": { "brand": "blocked-brand" } }
],
"should": [
  { "term": { "tags": { "value": "bestseller", "boost": 2 } } }
]
}
}
```

语义：

- **must**：必须匹配，通常参与评分。
- **filter**：必须匹配，不参与评分。
- **must_not**：必须不匹配，filter 语义。
- **should**：可用于可选召回或加分；是否必须命中受 `minimum_should_match` 和其他子句影响。

4.4 为什么结构化条件放 filter

状态、租户、时间范围、权限等条件不需要相关性评分。放在 filter 中可以：

- 避免无意义的评分计算。
- 表达更准确的业务意图。
- 对重复过滤条件利用 bitset/cache 等优化机会。

但不要机械地说“filter 一定被缓存”。是否缓存取决于查询形态、重复度、segment 状态和缓存策略。高基数、每次都不同的时间范围未必有缓存收益。

4.5 常见查询陷阱

wildcard 前导通配符

```
"wildcard": { "name": "*search*" }
```

前导通配符可能枚举大量词项，代价高。可选方案：

- 正确分词和 n-gram。
- `search_as_you_type`。
- 对机器生成的大字符串评估 wildcard 字段类型。
- 限制输入长度、字符集和超时。

script query

脚本查询逐文档执行，通常比原生查询昂贵。优先在写入阶段预计算字段，或使用 runtime field 做低频探索，不要把脚本当默认业务查询。

should 语义误判

当 bool 中没有 must/filter 时，默认至少命中一个 should；当存在 must/filter 时，should 可能只是加分。需要强制命中时显式设置：

```
"minimum_should_match": 1
```

第二部分：查询与相关性

5. Elasticsearch 如何计算相关性？BM25 应如何调优？

5.1 结论先行

Elasticsearch 对文本字段默认使用 BM25 相似度。它综合词频、逆文档频率和字段长度归一化计算相关性。调优不应只改 BM25 参数，而应先治理分词、字段权重、召回集合、业务特征和评估数据。

5.2 BM25 的直观理解

BM25 主要回答三个问题：

1. 查询词在当前文档中出现得更多，是否更相关？通常是，但收益会逐渐饱和。
2. 查询词在所有文档中越稀有，是否越有区分度？通常是。
3. 同样出现一次，短字段中的命中是否比超长字段更集中？通常是。

简化表达：

```
score ≈ IDF(term) × TF saturation × field-length normalization
```

关键参数：

- k1：控制词频饱和速度。
- b：控制字段长度归一化强度。

面试中无需推导完整公式，但应说明 BM25 不是简单“出现次数越多分越高”。

5.3 相关性问题的通常不先改 BM25

推荐排查顺序：

1. **Mapping 是否正确**：全文字段是否被错误映射成 keyword。
2. **分词是否正确**：品牌、型号、中文词、同义词是否被错误切分。
3. **查询结构是否合理**：是否把精确字段和全文字段混在一个 match 中。
4. **字段权重是否合理**：标题、品牌、类目、正文的重要性不同。
5. **召回与排序是否分层**：先召回，再用业务特征重排。
6. **是否有离线评估集**：没有标注数据时，调参往往只是主观感觉。

5.4 multi_match 和字段权重

```
GET products/_search
{
  "query": {
    "multi_match": {
      "query": "Apple 手机",
      "fields": [
        "title^4",
        "brand^3",
        "category_name^2",
        "description"
      ],
      "type": "best_fields",
      "tie_breaker": 0.2
    }
  }
}
```

```
}
}
```

标题和品牌被赋予更高 boost。boost 只是相对权重，不应无限放大；当分数被某个字段完全主导时，其他信号失去作用。

5.5 function_score 与业务排序

商品搜索常加入：销量、库存、转化率、上新时间、商家质量等特征。

```
GET products/_search
{
  "query": {
    "function_score": {
      "query": {
        "match": { "title": "Java 书籍" }
      },
      "functions": [
        {
          "field_value_factor": {
            "field": "sales_score",
            "modifier": "log1p",
            "missing": 0
          },
          "weight": 0.3
        },
        {
          "gauss": {
            "created_at": {
              "origin": "now",
              "scale": "30d",
              "decay": 0.5
            }
          },
          "weight": 0.2
        }
      ],
      "score_mode": "sum",
      "boost_mode": "sum"
    }
  }
}
```

生产注意：

- 销量等大数值必须归一化，否则会吞没文本分数。
- 不要在每个请求中执行昂贵脚本。
- 排序特征应有上下限，防止异常数据冲击结果。
- 需要 A/B 测试、NDCG、MRR、CTR、转化率等离线和在线指标。

5.6 explain 与 profile

explain 用于理解单个文档为什么得到该分数：

```
GET products/_explain/1001
{
  "query": {
```

```
"match": { "title": "Java 搜索" }
}
}
```

profile: true 用于分析查询各阶段耗时，但有明显额外开销，只应在排障或测试环境有控制地使用：

```
GET products/_search
{
  "profile": true,
  "query": {
    "match": { "title": "Java 搜索" }
  }
}
```

5.7 同义词的正确处理

同义词可能用于：

```
手机, 移动电话, smartphone
ES, Elasticsearch
```

常见选择：

- 索引时展开：写入成本高，词典变更可能需要 reindex。
- 查询时展开：词典更新灵活，但查询复杂度和召回规模可能增加。
- synonym_graph：更适合处理多词同义词和短语语义。

同义词不是越多越好。错误扩展会降低精度，例如“苹果”可能是水果或品牌，应结合类目、意图识别和字段上下文。

5.8 高频追问

为什么同一查询在不同分片上的分数可能有差异？

IDF 等统计可能基于各分片局部统计。通常分片数据足够大且均匀时影响有限；

dfs_query_then_fetch 可以先收集全局统计，但增加一次分布式往返和成本，不应默认开启。

为什么排序后 _score 不再重要？

如果显式按价格排序，结果主要按价格排列。需要综合相关性时，应将业务排序融入 score，或在第一阶段召回后进行 rescore/应用层重排。

6. 分片、副本和路由是什么？写入与查询请求如何执行？

6.1 结论先行

一个索引由若干主分片构成，每个主分片是一个独立 Lucene index；副本分片是主分片的复制，用于高可用和扩展搜索吞吐。主分片数量决定数据路由的基本并行度，创建后不能直接修改，只能通过 split、shrink 或 reindex 等方式改变。

6.2 routing 计算

默认情况下，文档根据 `_id` 计算路由：

```
routing_value = _id 或自定义 routing
shard = hash(routing_value) % number_of_primary_shards
```

同一个文档的写入、GET、更新和删除必须路由到同一主分片。

自定义 routing 可以把同一租户的数据放在同一分片：

```
PUT orders/_doc/10001?routing=tenant-42
{
  "tenant_id": "tenant-42",
  "amount": 199.00
}
```

查询时指定相同 routing，可以避免广播到全部分片：

```
GET orders/_search?routing=tenant-42
{
  "query": {
    "term": { "tenant_id": "tenant-42" }
  }
}
```

风险：大租户可能形成热点分片。routing 是性能优化，也是数据分布约束，必须在上线前评估倾斜。

6.3 写入流程

简化流程：

1. 客户端请求任意节点，该节点成为 coordinating node。
2. coordinating node 根据 routing 定位目标主分片。
3. 请求转发到主分片所在节点。
4. 主分片校验并执行操作，分配序列号。
5. 操作复制到 in-sync 副本。
6. 满足确认条件后返回客户端。

重要概念：

- 主分片负责确定操作顺序。

- 副本不是异步“随便复制”的缓存，而是复制协议的一部分。
- 副本故障会影响集群健康和可用性，但具体写入是否成功取决于 in-sync 集合、活动分片要求和故障时序。

6.4 查询流程：query then fetch

典型搜索分两阶段：

Query phase

- coordinating node 将查询发送到目标索引的每个相关分片副本。
- 每个分片本地执行查询，返回 top N 的文档 ID、分数和排序值。
- coordinating node 合并各分片结果，得到全局 top N。

Fetch phase

- coordinating node 向持有最终命中文档的分片请求 `_source` 或指定字段。
- 合并后返回客户端。

深分页成本高的原因由此可见：from=100000&size=20 时，每个分片都可能需要维护前 100020 条候选结果，再由协调节点合并。

6.5 副本能否线性提升查询性能

不一定。副本提供更多可选 shard copy，使并发搜索有机会分摊到更多节点，但前提是：

- 有足够数据节点承载副本。
- 查询并发足够高。
- CPU、磁盘、文件系统缓存和网络没有其他瓶颈。
- 分片数量和请求路由合理。

单次查询仍然只会访问每个 shard group 的一个 copy，增加副本通常不会让一条查询同时主副本上并行两份。

6.6 节点角色

生产集群常见角色：

- master-eligible：参与集群管理和主节点选举。
- data：存储分片并执行搜索、聚合和写入。
- ingest：执行 ingest pipeline。
- coordinating only：只负责请求分发和结果归并。
- remote_cluster_client、ml、transform 等特定角色。

小集群不应过早拆分角色。角色隔离适合规模足够大、负载边界明确的集群，否则会增加成本和故障面。

第三部分：容量、写入与分页

7. 主分片数量应如何规划？如何估算分片与集群容量？

7.1 结论先行

分片不是越多越好。分片太少会限制数据分布和并行度，分片太多会增加堆内元数据、线程调度、文件句柄、集群状态和查询扇出成本。最可靠的方法是基于真实数据和查询压测，目标是让单分片大小、数量、恢复时间和峰值负载都可控。

7.2 分片规划的四个约束

数据规模

单主分片目标大小 $\approx$ 主数据总量 / 主分片数

常见经验区间会把普通日志或内容分片控制在数十 GB 量级，但这不是硬规则。大分片减少元数据开销，却增加恢复、迁移和 merge 时间；小分片并行度高，却会制造碎片化。

节点数量

主分片和副本需要分布到不同节点。只有两个数据节点却设置两个副本，至少有一个副本无法分配，因为同一 shard group 的多个 copy 不应位于同一节点。

查询并行度

每个 shard 上的查询通常由单个线程执行。过大的分片可能使单分片查询延迟高；过多分片则使一次查询扇出到大量任务，协调和队列开销增加。

故障恢复目标

节点故障后需要恢复分片。分片越大，恢复单个分片越慢；分片太多，调度和小文件开销越大。应以 RTO 反推分片布局。

7.3 容量估算示例

假设：

- 每天写入原始日志 500 GB。
- ES 索引后膨胀系数按 1.2 预估。
- 保留 14 天热数据。
- 1 个副本。
- 预留 25% 磁盘安全空间。

主分片数据：

$500 \text{ GB} \times 1.2 \times 14 = 8.4 \text{ TB}$

包含副本：

$$8.4 \text{ TB} \times 2 = 16.8 \text{ TB}$$

考虑安全余量：

$$16.8 \text{ TB} / 0.75 \approx 22.4 \text{ TB}$$

如果每个数据节点可安全使用 2 TB，则至少需要约 12 个数据节点，实际还要考虑：

- 峰值写入和查询 CPU。
- merge 临时空间。
- 节点故障后的剩余容量。
- 冷热分层和快照策略。
- 字段数量、聚合复杂度与堆需求。

7.4 如何确定 rollover 条件

时序索引不建议固定“每天一个索引”而完全忽略流量波动。更合理的是基于：

- max_primary_shard_size
- max_docs
- max_age

做 rollover。这样高峰日不会产生超大分片，低流量日也不会产生大量极小索引。

7.5 过度分片的症状

- 集群状态更新慢，master 压力高。
- 节点重启和恢复时间长。
- 搜索线程池队列堆积。
- JVM 堆被 segment/shard 元数据占用。
- _cat/shards 数量巨大，单分片数据很小。
- 打开、关闭、创建索引耗时增加。

治理手段：

- Data Stream + rollover。
- 合理合并小索引或 reindex。
- 对只读索引 shrink。
- 删除无价值字段和索引。
- 避免为每个小租户创建大量独立索引。

7.6 面试追问

一开始主分片数设少了怎么办？

可以新建更多主分片的新索引并 reindex，也可在满足条件时 split。通常 alias + reindex 更通用、可控。

主分片数设多是否为未来扩容留余量？

过度预分片会立即付出资源成本。更推荐 rollover、模板、数据流和可控 reindex，而不是为未知未来创建数百个空分片。

8. from/size、search_after、PIT、scroll 应如何选择？

8.1 结论先行

- 小页码交互查询：from + size。
- 深分页和连续翻页：search_after + PIT。
- 批量导出和离线遍历：可根据版本与场景使用 PIT + search_after；scroll 仍可用于批处理，但官方不再推荐把 scroll 作为深分页 UI 方案。
- 需要随机跳到第 1000 页：Elasticsearch 天然不擅长，应重新设计产品交互或维护游标/业务快照。

8.2 from/size 的成本

```
GET products/_search
{
  "from": 100000,
  "size": 20,
  "sort": [
    { "created_at": "desc" },
    { "product_id": "asc" }
  ]
}
```

在分布式查询中，每个分片需要返回足够多的候选，由协调节点归并和丢弃前 100000 条。页码越深，CPU、堆和网络开销越大。

默认 index.max_result_window 常见为 10000。直接调大只是放开限制，不会消除算法成本。

8.3 search_after

第一页：

```
GET products/_search
{
  "size": 20,
  "query": { "match_all": {} },
  "sort": [
    { "created_at": "desc" },
    { "product_id": "asc" }
  ]
}
```

响应最后一条：

```
"sort": [1719630000000, "P10086"]
```

下一页：

```
GET products/_search
{
  "size": 20,
  "query": { "match_all": {} },
  "search_after": [1719630000000, "P10086"],
  "sort": [
    { "created_at": "desc" },

```

```
{ "product_id": "asc" }
]
```

要求：

- 每一页使用完全相同的 query 和 sort。
- sort 必须稳定，最后增加唯一 tie-breaker。
- 不能任意跳页，只能基于上一页游标前进。

8.4 为什么需要 PIT

分页过程中索引可能 refresh，新文档插入或旧文档更新会改变排序顺序，导致重复或漏数据。PIT 保存一个逻辑搜索视图：

```
POST products/_pit?keep_alive=2m
```

返回 PIT ID 后：

```
GET /_search
{
  "size": 20,
  "pit": {
    "id": "PIT_ID",
    "keep_alive": "2m"
  },
  "sort": [
    { "created_at": "desc" },
    { "_shard_doc": "asc" }
  ]
}
```

PIT 不是免费资源：它会延长旧 segment 的生命周期，并消耗文件句柄和磁盘空间。keep_alive 应按页间操作刷新，避免创建大量长时间 PIT。

8.5 scroll 的定位

scroll 会维护搜索上下文，适合批量处理历史数据，不适合作为用户实时翻页接口。问题包括：

- 长时间占用搜索上下文和旧 segment。
- 并发大量 scroll 会消耗资源。
- 不支持随机跳页。
- 新版本官方更推荐 PIT + search_after 处理深分页。

8.6 导出百万数据如何设计

不要让一个 HTTP 请求同步等待百万数据返回。可采用：

1. 创建异步导出任务。
2. 固定查询条件和 PIT/时间范围。
3. 使用 search_after 分批拉取，例如每批 1000-5000。
4. 流式写入对象存储。
5. 记录游标、进度和失败重试。
6. 限流并设置最大导出范围。
7. 完成后返回临时下载地址。

这样可以避免应用内存、ES 搜索上下文和网关连接被长时间占用。

9. 聚合为什么容易消耗内存？doc_values、fielddata 和全局序数是什么？

9.1 结论先行

全文检索依赖“词项到文档”的倒排结构，排序和聚合则经常需要“文档到字段值”的列式结构。

Elasticsearch 默认通过磁盘上的 doc values 支持 keyword、数值、日期等字段的排序和聚合；text 字段没有适合原值聚合的 doc values，开启 fielddata 会在 JVM 堆中构建结构，成本很高。

9.2 doc values

可以把 doc values 理解成按列存储的索引结构：

```
Doc1 -> brand=Apple, price=5999
Doc2 -> brand=Huawei, price=4999
Doc3 -> brand=Apple, price=6999
```

用于：

- sort
- terms/range/date histogram 等 aggregation
- script 中的字段访问
- 部分字段值返回

它主要驻留于磁盘并受文件系统缓存加速，减少 JVM 堆压力。

9.3 fielddata

text 字段被分词后没有一个天然“原始字符串值”可直接用于排序或 terms 聚合。开启：

```
"fielddata": true
```

会从倒排索引反向构建内存结构，可能占用大量堆并触发 fielddata circuit breaker。

正确方案通常是：

```
"title": {
  "type": "text",
  "fields": {
    "keyword": { "type": "keyword" }
  }
}
```

然后对 title.keyword 聚合。

9.4 terms 聚合与高基数

```
GET orders/_search
{
  "size": 0,
```

```

"aggs": {
  "top_users": {
    "terms": {
      "field": "user_id",
      "size": 100
    }
  }
}

```

高基数字段有数百万唯一值时，聚合可能消耗大量：

- 分片本地 bucket 内存。
- 协调节点归并内存。
- 网络传输。
- 全局序数构建时间。

优化方式：

- 避免对超高基数字段做无限 top N。
- 使用 composite aggregation 做可分页的全量桶遍历。
- 先过滤缩小数据范围。
- 调整 shard_size 时理解准确率与资源的权衡。
- 对频繁聚合的 keyword 评估 eager_global_ordinals。
- 预聚合、Transform 或离线数仓处理复杂报表。

9.5 global ordinals

每个 segment 可以把 keyword 词项映射为本地 ordinal。跨 segment 做 terms 聚合时，需要建立全局 ordinal 映射。默认可能在首次聚合时延迟构建，因此刚 refresh 后的第一次查询变慢。

对高频聚合字段可设置：

```
"eager_global_ordinals": true
```

这会把成本移动到 refresh 阶段，适合读多写少或查询延迟敏感的场景；写入频繁时会增加 refresh 开销。

9.6 composite aggregation

当需要遍历所有聚合桶时，不应把 terms size 设置成百万。composite aggregation 支持 after_key：

```

GET orders/_search
{
  "size": 0,
  "aggs": {
    "users": {
      "composite": {
        "size": 1000,
        "sources": [
          { "user": { "terms": { "field": "user_id" } } }
        ]
      }
    }
  }
}

```

下一页带上响应的 `after_key`。它适合离线遍历桶，不代表所有聚合都应改成 `composite`。

10. object、nested、flattened 有什么区别？如何避免 Mapping Explosion？

10.1 结论先行

普通 object 会把内部字段扁平化，数组对象之间的对应关系可能丢失；nested 将每个子对象作为隐藏 Lucene 文档保留关联，但查询、更新和存储成本更高；flattened 把任意键值对象作为单字段处理，适合未知 key 很多的场景，但查询能力受限。

10.2 object 的交叉匹配问题

文档：

```
{
  "users": [
    { "name": "Alice", "age": 20 },
    { "name": "Bob", "age": 40 }
  ]
}
```

普通 object 可能被扁平化为：

```
users.name = [Alice, Bob]
users.age  = [20, 40]
```

查询 “name=Alice AND age=40” 可能误匹配，因为字段之间的对象边界丢失。

10.3 nested

Mapping：

```
PUT people
{
  "mappings": {
    "properties": {
      "users": {
        "type": "nested",
        "properties": {
          "name": { "type": "keyword" },
          "age": { "type": "integer" }
        }
      }
    }
  }
}
```

查询：

```
GET people/_search
{
  "query": {
```

```

    "nested": {
      "path": "users",
      "query": {
        "bool": {
          "filter": [
            { "term": { "users.name": "Alice" } },
            { "term": { "users.age": 40 } }
          ]
        }
      }
    }
  }
}

```

此时不会误匹配。

nested 的代价：

- 每个 nested 对象是额外隐藏文档。
- 更新根文档会重写相关 Lucene 文档。
- nested query/aggregation 更复杂。
- 单文档 nested 对象数量受限制。

只有需要保持对象数组内部关联时才使用 nested。

10.4 flattened

适合：

```

"labels": {
  "os": "linux",
  "region": "us-west",
  "customer.custom.key.123": "value"
}

```

如果 key 数量不受控，动态映射会为每个 key 创建字段。flattened 把整个对象映射为一个字段，显著降低 Mapping 数量。

局限：

- 子值通常按 keyword 语义处理。
- 范围、数值类型和复杂分析能力有限。
- 不适合需要对每个子字段精确类型建模的核心业务数据。

10.5 Mapping Explosion

常见原因：

- 把用户自定义 JSON 的每个 key 自动映射成字段。
- 每个指标名、标签名或动态属性都成为新字段。
- 多租户共用索引，租户字段集合不断累加。
- 默认 string 同时创建 text + keyword 子字段。

症状：

- 超过 `index.mapping.total_fields.limit`，写入被拒绝。
- master 发布集群状态变慢。
- Kibana 字段列表卡顿。
- JVM 堆和请求体变大。

- Mapping 更新频繁，集群状态不稳定。

治理：

1. 上游规范字段命名，不允许把值当字段名。
2. 核心索引使用 `dynamic: strict`。
3. 动态区域使用 `dynamic template`、`flattened` 或 `enabled:false`。
4. 按业务域拆分索引，而不是无限共享一个 Mapping。
5. 只提高字段上限通常是延迟爆炸，不是根治。

11. 如何提高批量写入吞吐量，同时避免把集群写垮？

11.1 结论先行

提高写入吞吐的核心是：使用 Bulk 减少请求开销，逐步找到合理批次和并发，减少不必要的 refresh、副本和字段成本，并对 429、队列、indexing pressure、merge、磁盘和 GC 做闭环监控。优化必须以集群稳定为上限，而不是让客户端尽可能多发请求。

11.2 Bulk 大小如何确定

Bulk 请求：

```
POST _bulk
{ "index": { "_index": "products", "_id": "1" } }
{ "name": "Java" }
{ "index": { "_index": "products", "_id": "2" } }
{ "name": "Elasticsearch" }
```

没有一个适合所有集群的固定条数。测试方法：

1. 从 100-500 条或几 MB 开始。
2. 逐步增大批次，观察吞吐和延迟。
3. 当吞吐不再提升或堆/GC/429 明显增加时停止。
4. 生产保留安全余量。

不要只按条数，因为单文档可能从几百字节到几 MB。通常同时限制“最大条数 + 最大字节数”。

11.3 并发控制与背压

客户端应：

- 限制并发 Bulk 数量。
- 对 HTTP 429 使用指数退避和抖动。
- 设置最大重试次数和死信队列。
- 区分可重试错误与 Mapping/解析等永久错误。
- 记录每条 item 的失败，不能只看 Bulk HTTP 200。

Bulk API 整体返回成功不代表每个 item 成功，必须检查：

```
"errors": true
```

以及每个 action 的 status/error。

11.4 写入期间可调参数

对于离线首次导入：

- 暂时增大 refresh_interval，甚至在明确场景下设为 -1。
- 临时把副本数降为 0，导入后恢复副本。
- 使用自动生成 _id 可以减少部分查找成本；业务必须按 ID 幂等时不能盲目使用。
- 导入后等待副本恢复，再切换业务流量。

在线持续写入不应随意关闭副本或 refresh。可靠性与实时性要求优先。

11.5 Mapping 对写入的影响

- 不需要查询的字段设置 `index: false`。
- 不需要聚合的字段可评估关闭 `doc values`。
- 避免默认 `text + keyword` 双份索引。
- 不保存无用大字段或重复内容。
- 高频更新对象尽量拆分，避免每次重写超大 `_source`。
- 使用 `ingest pipeline` 时监控处理器耗时，复杂 `grok/script` 可能成为瓶颈。

11.6 热点和倾斜

如果所有事件使用同一个 `routing`，或者某个租户占据大部分写入，目标分片会成为热点：

- 单分片 CPU 高。
- `write` 线程池排队。
- 其他分片空闲。
- 总集群看似有资源，但吞吐无法提升。

解决方式：

- 使用均匀 `_id`。
- 对大租户拆分 `routing bucket`。
- 将热点租户放入独立索引。
- 增加主分片只能通过新索引、`split` 或 `reindex`，并非动态参数。

11.7 关键监控

- `indexing rate / indexing latency`
- `bulk request latency` 和失败率
- `429/rejected` 数量
- `write/search thread pool active、queue、rejected`
- `indexing pressure`
- `refresh/flush/merge time`
- `segment count`
- `disk utilization` 与 `IO latency`
- `JVM old-gen pressure` 和 `GC pause`
- `shard distribution` 与单分片文档数

第四部分：运维与故障排查

12. Elasticsearch 查询突然变慢，如何系统排查？

12.1 结论先行

查询变慢不能只看 DSL。应按“入口与范围 → 集群健康 → 节点资源 → 线程池与队列 → 分片与数据分布 → 查询计划 → Mapping 与 segment → 下游网络”逐层定位。第一目标是区分：单条坏查询、热点分片、资源饱和、后台任务竞争，还是集群状态异常。

12.2 第一步：确认问题边界

先回答：

- 所有查询慢，还是某类查询慢？
- P50、P95、P99 哪个变化？
- 从什么时候开始？是否伴随发版、数据增长、rollover、节点重启？
- 只在某个索引、租户、时间范围或节点上发生？
- ES 服务端 took 高，还是应用端总耗时高但 took 正常？

如果 ES 返回 took=50ms，应用观察 3s，重点应检查：

- 连接池等待。
- DNS、TLS、代理、网关和网络。
- 客户端反序列化。
- 应用线程池与 GC。

12.3 集群与分片检查

```
GET _cluster/health
GET _cat/nodes?v&s=cpu:desc
GET _cat/shards?v&s=state,index,shard
GET _cat/allocation?v
GET _nodes/stats
```

关注：

- red/yellow、unassigned shards。
- 单节点 CPU、heap、load、disk 异常。
- 分片是否集中到少数节点。
- 磁盘水位导致分片无法迁移。
- 节点是否频繁加入/离开。

12.4 线程池与任务

```
GET _cat/thread_pool/search?v
GET _cat/thread_pool/write?v
GET _tasks?detailed=true&actions=*
GET _nodes/hot_threads
```

搜索队列和 rejected 增加可能说明：

- 查询并发过高。
- 单查询扇出太多分片。
- 聚合、脚本、通配符等查询过重。

- CPU 被 merge、GC、向量构建或 ingest 抢占。

hot_threads 可帮助识别 CPU 在：

- Lucene 查询。
- global ordinals。
- segment merge。
- GC。
- snapshot/recovery。
- script 或正则。

12.5 查询级定位

Slow log

为目标索引设置适当阈值，记录 query/fetch 慢日志。阈值应按业务 SLA 设置，避免设置过低导致日志本身形成压力。

Profile API

```
GET products/_search
{
  "profile": true,
  "query": { ... },
  "aggs": { ... }
}
```

检查：

- 哪个 query 子句耗时最高。
- rewrite、create_weight、next_doc、advance 等阶段。
- aggregation collector 开销。
- 各 shard 耗时是否差异巨大。

Profile 会增加开销，不要对线上全量请求长期开启。

12.6 常见根因和修复

根因	症状	处理方向
深分页	from 很大，协调节点内存高	search_after + PIT
查询分片过多	单索引小分片很多	rollover/shrink/reindex/限定索引范围
高基数聚合	heap 高、breaker	缩小范围、composite、预聚合
wildcard/regexp	CPU 高	改字段与分词、限制模式
text fielddata	fielddata 飙升	keyword multi-field
segment 太多	查询、文件句柄、merge 高	调整 refresh、写入方式，等待合并
cache miss	首次查询慢	预热或稳定路由，避免频繁 refresh
热点分片	某节点/分片持续慢	调整 routing、拆索引、再平衡
_source 很大	fetch phase 慢	source filtering、fields、拆大字段

根因	症状	处理方向
大量脚本	CPU 高	写入预计算、减少 runtime/script

12.7 止损优先级

线上故障先止损：

1. 限流或关闭高成本功能。
2. 缩短查询时间范围、降低聚合 size。
3. 对异常租户隔离。
4. 取消失控任务。
5. 临时扩容或增加副本分摊读流量。
6. 根因修复后再逐步恢复。

不要在高压时同时大量调整分片、force merge、清缓存和重启节点，这些动作可能放大抖动。

13. 集群变成 yellow 或 red，未分配分片如何定位和恢复？

13.1 结论先行

- green: 主分片和副本都已分配。
- yellow: 所有主分片可用，但至少一个副本未分配，数据仍可服务但容灾能力下降。
- red: 至少一个主分片未分配，部分数据不可用。

故障核心不是“执行 reroute”，而是用 allocation explain 找出分配决策被哪个 decider 阻止。

13.2 基础命令

```
GET _cluster/health?level=indices
GET _cat/shards?v&h=index,shard,prirep,state,unassigned.reason,node
GET _cluster/allocation/explain
```

指定分片：

```
POST _cluster/allocation/explain
{
  "index": "orders-000123",
  "shard": 2,
  "primary": false
}
```

响应会说明：

- 是否因磁盘水位拒绝。
- allocation filter 是否不匹配。
- 节点角色或数据 tier 是否缺失。
- 同一 shard copy 已在该节点。
- 节点数不足以放置副本。
- recovery 限流或延迟分配。

13.3 常见原因

节点数不足

1 个数据节点，索引设置 1 个副本时，副本不能和主分片位于同一节点，因此 yellow。

处理：增加节点或在可接受风险下把副本降为 0。

磁盘水位

磁盘达到 low/high/flood stage 时，ES 会限制分配，严重时对索引加只读保护。

处理：

- 删除过期数据或扩容。
- 修复 ILM/rollover。
- 等磁盘降到安全范围后确认只读块是否解除。

- 不建议长期提高水位阈值掩盖容量问题。

allocation awareness / filter 错误

例如要求副本跨 zone，但某个 zone 没有节点；或索引被固定在不存在的 tier。

处理：补齐节点或修正 allocation 规则。

节点离线或 shard copy 损坏

主分片丢失且没有可用 in-sync copy 时，可能 red。优先：

- 恢复原节点。
- 从 snapshot 恢复。
- 从上游主库或事件重放重建。

allocate_stale_primary 或 allocate_empty_primary 可能造成数据丢失，只能在理解损失并获得业务授权后使用。

13.4 为什么反复重启可能更糟

节点重启会触发：

- shard recovery。
- 文件系统缓存冷却。
- 集群状态变化。
- 大量网络和磁盘复制。

如果根因是磁盘满、Mapping 爆炸或查询过载，重启不能解决，反而可能让可用副本减少。应先收集证据，再做最小动作。

13.5 恢复后的验证

- 集群健康是否恢复并稳定一段时间。
- shard 是否均衡分布。
- recovery 是否完成。
- 磁盘水位和只读块是否正常。
- 应用错误率和查询延迟是否恢复。
- snapshot 最近一次是否成功。
- 是否需要修正容量、ILM 和告警阈值。

14. JVM 堆、文件系统缓存、GC 和熔断器应如何理解？

14.1 结论先行

Elasticsearch 性能不仅依赖 JVM 堆，也高度依赖操作系统文件系统缓存。堆过小会频繁 GC 和触发 breaker，堆过大则压缩文件缓存并增加 GC 暂停。现代版本通常建议优先使用自动堆大小，手工配置时必须为 OS 留出足够内存。

14.2 哪些数据在堆中

典型堆占用：

- 集群状态和 Mapping。
- 查询中间结果和聚合 bucket。
- fielddata、global ordinals 的部分结构。
- request cache/query cache 元数据。
- indexing buffer。
- 网络请求和反序列化对象。
- shard/segment 相关对象。

Lucene segment 文件、doc values 等大量数据依赖 mmap/文件系统缓存，并不全部装入 JVM 堆。

14.3 为什么堆不能占满物理内存

假设 64 GB 机器把 60 GB 都给 JVM：

- OS 文件缓存空间不足。
- Lucene 读取频繁落到真实磁盘。
- 搜索延迟变高。
- 大堆 GC 周期更长。
- 系统还需要给进程、网络、线程栈和 native memory 留空间。

传统经验经常提到堆不超过物理内存约一半和 compressed oops 边界，但现代 ES 支持自动 sizing。

面试应强调：不要死背 31 GB；优先遵循当前版本自动配置和实际监控。

14.4 GC 观察

健康堆通常呈锯齿形。应关注：

- old-gen memory pressure 的持续水平。
- old GC 次数和暂停时间。
- GC 后堆是否能明显下降。
- 节点是否因长时间 stop-the-world 被集群移除。

持续高于约 85% 的 JVM memory pressure 是官方排障文档建议关注的风险信号，但实际告警要结合版本、负载和持续时间。

14.5 Circuit Breaker

熔断器用于在预计内存消耗超过阈值时拒绝请求，保护节点避免 OOM。常见 breaker：

- parent breaker
- request breaker
- fielddata breaker
- in-flight requests breaker
- script compilation 等相关限制

出现 `circuit_breaking_exception` 时，不应第一反应就是调高阈值。先定位：

- 高基数聚合。
- 过大的 terms size。
- text fielddata。
- 并发大请求。
- 深分页。
- Mapping/分片过多。
- 应用重试风暴。

breaker 是保护机制，不是根因。

14.6 OOM 排查

1. 保存日志、GC 日志、节点统计和 heap dump。
2. 查看是否发生 breaker、rejected、长 GC。
3. 用 MAT/YourKit 分析最大对象和 GC root。
4. 检查 fielddata、query cache、聚合、Mapping 和 shard 数。
5. 判断是持续泄漏、瞬时大请求还是容量不足。
6. 先限流和扩容止损，再优化查询或 Mapping。

直接清缓存或重启可以短暂恢复，但会丢失根因证据并造成缓存冷启动。

第五部分：迁移、一致性与生命周期

15. Mapping 不能直接修改时，如何使用 alias + reindex 实现无停机迁移？

15.1 结论先行

使用版本化物理索引和稳定别名：业务只访问 alias；创建新索引、reindex 历史数据、同步迁移期间增量、校验后原子切换 alias。关键难点不是 reindex 命令，而是迁移期间的增量一致性和回滚设计。

15.2 基础流程

当前：

```
products-read -> products-v1
products-write -> products-v1
```

创建新索引：

```
PUT products-v2
{
  "settings": { ... },
  "mappings": { ... }
}
```

历史迁移：

```
POST _reindex?wait_for_completion=false
{
  "source": { "index": "products-v1" },
  "dest": { "index": "products-v2" }
}
```

校验后原子切换：

```
POST _aliases
{
  "actions": [
    { "remove": { "index": "products-v1", "alias": "products-read" } },
    { "add": { "index": "products-v2", "alias": "products-read" } },
    { "remove": { "index": "products-v1", "alias": "products-write" } },
    { "add": { "index": "products-v2", "alias": "products-write", "is_write_index": true } }
  ]
}
```

alias actions 在一次请求中原子应用，客户端不会看到一半切换状态。

15.3 增量数据如何处理

仅执行一次 reindex 会漏掉迁移期间新写入或更新的数据。常见方案：

双写

应用同时写 v1 和 v2。

优点：实时。

问题：

- 两次写入可能部分失败。
- 必须有重试、对账和幂等。
- 业务代码复杂。

CDC/MQ 重放

主库变更通过事件写入 v1/v2，迁移时记录 offset，历史 reindex 后重放指定时间点之后的事件。

优点：一致性链路清晰，适合本来就有搜索同步平台的系统。

短暂停写

适合数据量小、可以接受短维护窗口的系统。暂停写入 → reindex 增量 → 切换 → 恢复写入。

15.4 校验不能只比 count

需要：

- 文档总数和按业务维度计数。
- 随机抽样 _source。
- 关键字段缺失率。
- 业务查询 top N 对比。
- 聚合结果对比。
- 写入延迟和错误率。
- Mapping、settings、alias 检查。

reindex 默认不会自动复制全部 settings、模板和 alias，必须显式创建目标索引。

15.5 回滚

- 切换前保留 v1。
- 切换后观察一段时间，不立即删除旧索引。
- 读 alias 可以快速切回。
- 如果写入已只进入 v2，回滚必须考虑 v2 新增数据如何同步回 v1。

真正的“无停机”不仅是 alias 原子切换，还包括双向增量策略和可执行回滚预案。

16. Elasticsearch 如何处理写入一致性、并发更新和版本冲突？

16.1 结论先行

Elasticsearch 在单文档级别提供原子写入，并使用 `_seq_no` 与 `_primary_term` 进行乐观并发控制。搜索是近实时的，副本用于可用性和复制，但 Elasticsearch 不提供跨多文档的传统 ACID 事务。

16.2 丢失更新示例

两个线程读取同一个库存文档：

```
stock = 10
```

A 减 1 写回 9，B 也基于旧值减 1 写回 9，丢失一次扣减。

读取文档时获取：

```
"_seq_no": 362,  
"_primary_term": 2
```

更新时：

```
PUT inventory/_doc/sku-1?if_seq_no=362&if_primary_term=2  
{  
  "stock": 9  
}
```

若期间已有其他写入，返回 409 conflict，客户端重新读取并重试或走业务冲突处理。

16.3 Update API 是否没有并发问题

Update API 在分片侧执行“取文档、运行脚本、重新索引”，减少客户端往返，但仍可能遇到冲突。可以配置 `retry_on_conflict`：

```
POST inventory/_update/sku-1?retry_on_conflict=3  
{  
  "script": {  
    "source": "ctx._source.stock -= params.count",  
    "params": { "count": 1 }  
  }  
}
```

风险：库存不能为负、订单幂等等业务约束不能只靠盲目重试。高并发库存扣减更适合数据库条件更新、Redis Lua、队列串行化或专门库存服务，ES 应作为查询副本。

16.4 external version

当上游系统有单调递增版本号时，可以用 external version 控制旧事件覆盖新数据。适合 CDC 乱序事件，但版本必须满足严格语义。

更常见的做法是：

- 使用数据库 binlog offset、业务版本或更新时间。
- 消费端比较版本。
- 对删除事件同样保留 tombstone/版本语义。

仅比较时间戳可能受时钟偏差和同毫秒更新影响。

16.5 写入成功后搜索不到是否数据丢失

通常不是。可能还未 refresh。区分：

- Index API 已确认：写入复制和持久性满足当前配置。
- Search API 可见：等待 refresh。
- GET by ID：通常可实时读取。

因此对写后读：

- 需要按 ID 确认时使用 GET。
- 需要搜索命中时使用 refresh=wait_for 或业务等待。
- 不要无限重试写入，必须使用稳定 _id 保证幂等。

16.6 多文档事务怎么办

例如订单和订单明细必须原子一致，ES 不适合作为事务协调器。方案：

- 事务在主数据库完成。
- 提交后通过 outbox/CDC 发送事件。
- ES 以最终一致方式建立搜索文档。
- 搜索文档可以反范式包含完整展示数据。
- 通过重放和对账修复不一致。

17. 日志与时序数据为什么适合 Data Stream + ILM? 如何设计生命周期?

17.1 结论先行

日志、指标和追踪通常是带时间戳、持续追加、按时间范围查询、旧数据价值逐渐下降的数据。Data Stream 提供稳定逻辑名称和自动管理的 backing indices, ILM 根据大小、年龄等条件 rollover, 并在 hot、warm、cold、frozen、delete 阶段自动执行迁移和清理。

17.2 Data Stream 结构

```
logs-app-prod (数据流)
├── .ds-logs-app-prod-2026.06.01-000001
├── .ds-logs-app-prod-2026.06.15-000002
└── .ds-logs-app-prod-2026.06.29-000003  <- write index
```

应用始终写入 logs-app-prod, 无需知道当前 backing index。

必要条件:

- 文档包含 @timestamp。
- composable index template 声明 data_stream。
- 模板包含 Mapping、settings 和 lifecycle 策略。

17.3 ILM 示例

```
PUT _ilm/policy/logs-policy
{
  "policy": {
    "phases": {
      "hot": {
        "actions": {
          "rollover": {
            "max_primary_shard_size": "40gb",
            "max_age": "1d"
          }
        }
      },
      "warm": {
        "min_age": "7d",
        "actions": {
          "forcemerge": { "max_num_segments": 1 }
        }
      },
      "delete": {
        "min_age": "30d",
        "actions": { "delete": {} }
      }
    }
  }
}
```

示例只是说明结构。force merge 是否合适、何时迁移 tier、是否使用 searchable snapshot，需要根据查询频率、成本和版本功能测试。

17.4 为什么 rollover 优于固定日期索引

固定每天索引在流量变化时会出现：

- 高峰日单分片过大。
- 低峰日大量小分片。
- 恢复和查询性能不稳定。

rollover 根据 shard size + age 组合触发，更接近资源目标。

17.5 Data Stream 的限制与选择

Data Stream 更适合 append-only。虽然可以直接操作 backing index 进行更新或删除，但如果业务频繁使用同一 _id 覆盖更新，普通索引 + write alias 可能更自然。

选择：

- 日志、指标、追踪：Data Stream。
- 商品、用户、订单搜索副本：版本化索引 + alias。
- 高频 update/delete 的时序数据：评估 alias rollover 或重新建模。

17.6 日志平台设计要点

- ECS 或统一字段规范，避免不同团队字段类型冲突。
- namespace/dataset/environment 作为低基数维度。
- 不允许任意 JSON key 直接映射。
- 日志级别、服务名、trace_id 使用 keyword。
- message 用 text；原始超大 payload 可放对象存储，只存引用。
- 通过 ingest pipeline 做必要解析，但复杂清洗可在 Logstash/Flink 上完成。
- ILM/DSL、磁盘水位、ingest lag、失败存储和快照必须监控。

18. Snapshot 是否等于实时备份？如何设计备份与灾难恢复？

18.1 结论先行

Snapshot 是 Elasticsearch 官方的数据备份和恢复机制，基于 segment 做增量存储，但它不是某个绝对瞬间的全局事务快照，也不能替代跨区域容灾、上游主数据和恢复演练。Snapshot 期间每个 shard 的数据视图可能位于开始和结束时间之间。

18.2 Snapshot 的特点

- 增量：复用仓库中已有的不可变 segment。
- 支持索引、数据流和部分集群状态。
- 可通过 SLM 自动创建和保留。
- 仓库可使用 S3、GCS、Azure、共享文件系统等。
- 恢复受 Elasticsearch 和索引创建版本兼容性约束。

不要直接复制 data/ 目录作为备份。运行中的 Lucene 文件和集群元数据需要一致性协议，文件级拷贝可能不可恢复。

18.3 备份策略

定义：

- RPO：最多可以丢多少时间的数据。
- RTO：故障后多久必须恢复。

示例：

- 每 30 分钟增量 snapshot，RPO 约 30 分钟。
- 快照保留 30 天。
- 关键索引复制到异地仓库。
- 每季度执行恢复演练并记录实际 RTO。
- MySQL 主库和 Kafka 日志保留足够时间，可重建搜索索引。

搜索副本的最佳 DR 常常是：

Snapshot 快速恢复 + 主库/事件流可重建

18.4 恢复注意事项

- 目标集群版本必须兼容。
- 不能直接恢复覆盖同名打开索引；可关闭、删除或 rename on restore。
- 恢复前确认磁盘和节点容量。
- 数据流恢复需要匹配的数据流模板。
- 只恢复数据不一定恢复应用所需的模板、pipeline、用户和安全配置，必须明确 feature state 和配置备份范围。

18.5 Snapshot 成功不等于备份可用

必须监控：

- 最近成功时间。
- snapshot state 和失败 shard。
- 仓库可读写性。
- 保留策略是否执行。
- 实际恢复测试。
- 跨区域访问权限和加密密钥。

没有恢复演练的备份，只能算“可能存在的数据副本”。

第六部分：现代搜索与系统设计

19. 向量检索、BM25 和混合检索如何选型？

19.1 结论先行

BM25 擅长关键词、精确术语、型号和可解释文本相关性；向量检索擅长语义相似和同义表达；混合检索通常同时保留两者的召回优势，再用 RRF、线性融合或重排模型合并。向量检索不是 BM25 的无条件替代品。

19.2 向量检索基本链路

文档文本 -> embedding model -> dense vector -> 建立向量索引
 用户查询 -> embedding model -> query vector -> kNN 召回

Elasticsearch 可在 dense_vector 等字段上执行近似 kNN。近似算法通常以召回精度换取速度，常见 HNSW 结构需要额外内存和构建成本。

19.3 适用场景

场景	BM25	向量
SKU/型号精确搜索	强	弱
法律条文关键词	强	中
“适合雨天跑步的鞋” 语义搜索	中	强
拼写、同义表达	中	强
可解释性	强	较弱
新鲜实时写入成本	较低	embedding + 图构建成本

19.4 混合检索

方案一：分别召回 BM25 和 kNN，使用 RRF 按排名融合。RRF 不要求两类分数在同一数值范围，适合 BM25 无界分数和向量相似度分数难以直接归一化的情况。

方案二：线性融合：

$$\text{final_score} = \alpha \times \text{normalized_bm25} + \beta \times \text{normalized_vector} + \gamma \times \text{business_score}$$

需要可靠归一化和权重评估。

方案三：第一阶段大规模召回，第二阶段使用 cross-encoder/reranker 对前几十或几百条精排。效果好但增加模型推理延迟和成本。

19.5 过滤与向量检索

商品搜索通常必须加：

- status=online
- tenant/region
- category
- price range
- permission

过滤应尽量进入 kNN 查询的过滤路径，避免先取很小的向量 top K 再由应用过滤，否则可能没有足够结果。

19.6 性能与容量

关注：

- embedding 维度。
- 向量数量和索引大小。
- HNSW 参数和量化方式。
- num_candidates 与 recall/latency。
- 同时写入时小 segment 对 kNN 查询的影响。
- 模型服务延迟、批量 embedding 和缓存。
- 向量更新会产生新文档版本和 merge 成本。

19.7 面试追问

为什么不能只用向量检索商品型号？

型号的语义向量可能把相似字符或相关产品混在一起，而用户通常要求精确命中。应使用 keyword/term 或 BM25 精确特征，并在混合召回中提高精确项权重。

RAG 中 ES 的角色是什么？

存储文档分块、元数据和向量，执行关键词、语义和权限过滤召回；最终答案仍由上层模型生成。检索质量、分块策略、权限和引用可追溯性比“是否用了向量”更关键。


```
{ "name": "storage", "value": "256GB" }
],
"updated_at": "2026-06-29T10:00:00Z"
}
```

字段设计:

- product_id、品牌、状态、类目: keyword。
- title: text + keyword/拼音/前缀等必要 multi-fields。
- price: long 或 scaled_float。
- attributes: 属性集合稳定时显式字段; 需要保持属性名和值关联时 nested; 任意扩展可用 flattened, 但过滤能力需评估。
- 排序特征: 写入时预计算并限制范围。
- 超大详情不放搜索文档, 避免 fetch phase 变慢。

20.4 分片和索引版本

- 使用 products-v202606 版本化物理索引。
- products-read 和 products-write alias 解耦业务。
- 通过压测确定主分片数, 使单分片大小、QPS 和恢复时间符合目标。
- 1 个副本起步, 跨可用区分配。
- 热点品牌/租户不能使用会倾斜的 routing; 如需租户 routing, 应对大租户做独立索引或 routing bucket。

20.5 数据同步一致性

事件包含:

```
product_id, operation, business_version, changed_fields, timestamp
```

消费者:

- 按 product_id 分区, 保证单商品有序。
- 使用稳定 _id=product_id 保证重复消费幂等。
- 比较 business_version, 拒绝旧事件覆盖新状态。
- Bulk item 级检查错误。
- 永久错误进入 DLQ, 告警并支持修复重放。
- 定期按数据库与 ES 做抽样、计数和关键字段对账。

上下架 5 秒 SLA 由以下链路共同保障:

```
CDC 延迟 + Kafka lag + 消费处理 + Bulk 等待 + refresh interval
```

不能只修改 refresh interval。

20.6 查询设计

```
GET products-read/_search
{
  "size": 20,
  "track_total_hits": false,
  "_source": ["product_id", "title", "brand_name", "price"],
  "query": {
    "bool": {
      "must": [
        {
          "multi_match": {
            "query": "iphone 256g 黑色",
```

```

        "fields": ["title^4", "brand_name^3", "category_name^2"]
      }
    },
    ],
    "filter": [
      { "term": { "status": "ONLINE" } },
      { "range": { "price": { "lte": 1000000 } } }
    ]
  }
},
"sort": [
  { "_score": "desc" },
  { "sales_score": "desc" },
  { "product_id": "asc" }
]
}

```

优化：

- 不需要精确总数时 track_total_hits:false 或设置上限。
- source filtering 只返回列表页字段。
- facets 聚合限制 size 和属性范围。
- 深翻页使用 PIT + search_after。
- 热门无个性化查询可做短 TTL 缓存，但必须把租户、区域、权限和排序纳入 key。

20.7 搜索建议

根据需求选择：

- 热门词：离线/流式统计，Redis 或专门索引。
- 前缀补全：completion suggester 或 search_as_you_type。
- 拼写纠错：term/phrase suggester 或查询理解服务。
- 个性化建议：用户画像与行为系统，不应让 ES 单独承担全部推荐逻辑。

建议接口必须限长、限字符和限频，防止每个键盘输入触发昂贵查询。

20.8 高可用和降级

- 至少 3 个 master-eligible 节点，数据节点跨可用区。
- 查询服务设置超时、熔断和并发上限。
- ES 慢时降级：关闭高亮/聚合/个性化，缩短时间和结果数，返回缓存或类目页。
- 搜索不可用不影响商品详情和下单核心链路。
- 定期 snapshot，并保证主库事件可重建索引。
- 索引迁移使用 alias + reindex + 增量同步。

20.9 可观测性

业务指标：

- 搜索 QPS、P50/P95/P99。
- 零结果率、点击率、转化率。
- 查询词分布、热门 filter。
- 索引延迟和事件积压。

ES 指标：

- 节点 CPU、heap、GC、disk、IO。
- search/write queue 和 rejected。

- shard size、segment count、merge time。
- query slow log、breaker、429。
- cluster health、unassigned shards、snapshot 状态。

20.10 方案取舍

- 更短 refresh 提高可见性，但降低写入吞吐。
- 更多副本提升读取冗余和并发，但增加写入与存储成本。
- nested 保持属性关联，但增加隐藏文档和查询成本。
- 实时价格库存放 ES 可以减少服务调用，但一致性和更新压力高；常见做法是 ES 召回商品，再批量查询价格库存服务。
- 向量搜索提升语义召回，但必须保留型号、品牌等精确检索并控制模型成本。

附录 A：常用排障 API 速查

集群与节点

```
GET _cluster/health
GET _cluster/state/metadata?filter_path=metadata.cluster_coordination
GET _cat/nodes?v
GET _nodes/stats
GET _nodes/hot_threads
```

分片与分配

```
GET _cat/indices?v&s=store.size:desc
GET _cat/shards?v&s=state,index
GET _cat/allocation?v
GET _cluster/allocation/explain
GET _cat/recovery?v&active_only=true
```

线程池与任务

```
GET _cat/thread_pool/search?v
GET _cat/thread_pool/write?v
GET _tasks?detailed=true&actions=*
POST _tasks/<task_id>/_cancel
```

Mapping 与字段

```
GET products/_mapping
GET products/_field_caps?fields=*
POST products/_analyze
{
  "field": "title",
  "text": "Elasticsearch 面试题"
}
```

查询分析

```
GET products/_search
{
  "profile": true,
  "query": { ... }
}

GET products/_explain/<id>
{
  "query": { ... }
}
```

Segment 与缓存

```
GET _cat/segments?v
GET _nodes/stats/indices/segments,fielddata,query_cache,request_cache
GET _cat/fielddata?v
```

ILM 与快照

```
GET products-*/_ilm/explain
GET _snapshot
GET _snapshot/<repo>/_all
GET _slm/status
```

附录 B：面试官高频连环追问

1. 为什么主分片数创建后不能直接改？

默认 routing 的模数依赖主分片数量，直接修改会改变文档到分片的映射；现有数据必须重新路由。因此需要 split、shrink 或 reindex。

2. 副本数可以动态修改吗？

可以。修改后集群会分配或移除副本，但会产生恢复和网络 IO，应在容量可控时操作。

3. Elasticsearch 写入是覆盖还是追加？

从 API 语义看，同 _id 可以覆盖更新；从 Lucene 视角是标记旧文档删除并写入新文档。

4. 删除文档后磁盘为什么没有立即下降？

删除先成为 segment 的删除标记，后续 merge 才真正回收。强制 merge 活跃索引风险较大。

5. 为什么分片多会使查询变慢？

一次查询要向更多 shard copy 分发，产生更多线程任务、网络响应、优先队列和归并工作；小分片也增加 segment 和元数据开销。

6. 查询缓存和请求缓存有什么区别？

查询缓存更偏向 filter 结果的可复用 bitset；shard request cache 缓存分片级完整结果，常见于 size:0 聚合，并在 refresh 或 Mapping 更新时失效。不要把缓存当成错误 DSL 的补救措施。

7. track_total_hits:false 有什么意义？

避免为了精确总数遍历所有命中，列表页通常只关心是否有更多结果。需要显示准确总数时才承担对应成本。

8. terminate_after 能否保证全局只查 N 条？

它通常在每个 shard 收集到指定数量后提前终止，不等于全局精确限制，也可能影响结果完整性。

9. 为什么不要把每个租户建成一个索引？

大量小租户会制造海量索引和分片。可共享索引加 tenant_id 过滤；超大或高价值租户再独立索引，形成分层方案。

10. 为什么 ES 的聚合不能完全替代数仓？

ES 适合交互式过滤与聚合，但复杂多表关联、长期全量扫描、财务级精确报表和离线计算更适合 OLAP/数仓。应按查询类型分工。

附录 C：容量与性能检查清单

上线前

- 数据量、日增量、保留周期和副本数是否量化。
- 主分片数量是否经过真实数据压测。
- Mapping 是否显式，动态字段是否受控。
- 中文分词、同义词、品牌和型号是否有测试集。
- Bulk 批次、并发和退避是否压测。
- 查询 P99、聚合和深分页是否符合 SLA。
- 节点故障后剩余容量是否仍可服务。
- 快照仓库和恢复流程是否演练。
- 版本化索引和 alias 是否准备。

运行中

- cluster health 与 unassigned shard。
- JVM memory pressure、GC 和 breaker。
- CPU、load、磁盘水位与 IO latency。
- thread pool queue/rejected。
- segment count、merge、refresh、flush。
- 单分片大小和热点倾斜。
- 搜索慢日志和异常 DSL。
- 写入 lag、Bulk item 失败和 DLQ。
- ILM/rollover 与 snapshot 成功率。

故障处理

- 先确认影响面和时间线。
- 保留日志、统计和 hot threads。
- 先限流降级，不盲目重启。
- allocation explain 查明未分配原因。
- breaker/429 是保护信号，先找重请求。
- 修复后进行容量、告警和演练复盘。

附录 D：官方参考资料

以下资料均来自 Elastic 官方文档，检索时间为 2026 年 6 月。由于 Elasticsearch 版本演进较快，升级或落地前应再次核对目标版本文档。

1. Elasticsearch 下载与当前版本: <https://www.elastic.co/downloads/elasticsearch>
2. Elasticsearch Release Notes: <https://www.elastic.co/docs/release-notes/elasticsearch>
3. Translog settings: <https://www.elastic.co/docs/reference/elasticsearch/index-settings/translog>
4. Refresh parameter: <https://www.elastic.co/docs/reference/elasticsearch/rest-apis/refresh-parameter>
5. Merge settings: <https://www.elastic.co/docs/reference/elasticsearch/index-settings/merge>
6. Mapping 与字段类型: <https://www.elastic.co/docs/manage-data/data-store/mapping>
7. text field: <https://www.elastic.co/docs/reference/elasticsearch/mapping-reference/text>
8. keyword field: <https://www.elastic.co/docs/reference/elasticsearch/mapping-reference/keyword>
9. doc values: <https://www.elastic.co/docs/reference/elasticsearch/mapping-reference/doc-values>
10. Dynamic templates: <https://www.elastic.co/docs/manage-data/data-store/mapping/dynamic-templates>
11. Mapping Explosion: <https://www.elastic.co/docs/troubleshoot/elasticsearch/mapping-explosion>
12. Nested field: <https://www.elastic.co/docs/reference/elasticsearch/mapping-reference/nested>
13. Paginate search results: <https://www.elastic.co/docs/reference/elasticsearch/rest-apis/paginate-search-results>
14. Search shard routing: <https://www.elastic.co/docs/reference/elasticsearch/rest-apis/search-shard-routing>
15. BM25 similarity: <https://www.elastic.co/docs/reference/elasticsearch/mapping-reference/similarity>
16. Indexing pressure: <https://www.elastic.co/docs/reference/elasticsearch/configuration-reference/indexing-pressure-settings>
17. Tune for indexing speed: <https://www.elastic.co/docs/deploy-manage/production-guidance/optimize-performance/indexing-speed>
18. Tune for search speed: <https://www.elastic.co/docs/deploy-manage/production-guidance/optimize-performance/search-speed>
19. Size your shards: <https://www.elastic.co/docs/deploy-manage/production-guidance/optimize-performance/size-shards>
20. Optimistic concurrency control: <https://www.elastic.co/docs/reference/elasticsearch/rest-apis/optimistic-concurrency-control>
21. Circuit breaker settings: <https://www.elastic.co/docs/reference/elasticsearch/configuration-reference/circuit-breaker-settings>
22. High JVM memory pressure: <https://www.elastic.co/docs/troubleshoot/elasticsearch/high-jvm-memory-pressure>
23. Index lifecycle management: <https://www.elastic.co/docs/manage-data/lifecycle/index-lifecycle-management>
24. Data Stream rollover: <https://www.elastic.co/docs/manage-data/lifecycle/index-lifecycle-management/rollover>
25. Reindex API: <https://www.elastic.co/docs/reference/elasticsearch/rest-apis/reindex-indices>
26. Snapshot and restore: <https://www.elastic.co/docs/deploy-manage/tools/snapshot-and-restore>
27. Reciprocal Rank Fusion: <https://www.elastic.co/docs/reference/elasticsearch/rest-apis/reciprocal-rank-fusion>
28. Tune approximate kNN search: <https://www.elastic.co/docs/deploy-manage/production-guidance/optimize-performance/approximate-knn-search>

结语

高级 Elasticsearch 面试的分水岭，不是能背出多少 API，而是能否把搜索效果、写入吞吐、分片容量、数据一致性、故障恢复和业务成本放在同一个系统中权衡。

一个成熟的回答通常会体现以下能力：

- 能区分 Lucene 原理、Elasticsearch 分布式机制和业务架构。
- 不把经验参数当绝对规则，而是强调压测与 SLA。
- 能解释正常链路，也能说明失败路径、止损和恢复。
- 知道 Elasticsearch 的优势，也知道哪些问题应该交给数据库、缓存、消息队列或数仓。
- 能把查询、Mapping、分片、JVM 和磁盘问题串成完整排障链路。